



CompSci 230

Object Oriented Software Development

Classes and Objects



Review Quizzes

```
ArrayList<String> recVehicles = new ArrayList<String>();  
recVehicles.add("car");  
recVehicles.add("truck");
```

- ▶ Consider the above code fragment, which is the correct syntax for placing the string "boat" into an ArrayList name `recVehicles` in position 3 (index 2) for the first time?

- a) `recVehicles.set(3, "boat");`
b) `recVehicles.add(2, "boat");` ✓

car	truck	
-----	-------	--

- ▶ Which of the following statements correctly stores the value that is the 5th element in an ArrayList of integers, `intArrList<Integer>`, into integer variable `currNum`?

- a) `currNum = intArrList.get(5);`
b) `currNum = intArrList[4];`
c) `currNum = intArrList.get(4);` ✓
d) `currNum = intArrList.elementAt(4);`



Review Quizzes

- ▶ Which of the following statements correctly change the value of the 3rd (index: ?) element in an array to 10?
 - a) `myarray.add(10);`
 - b) `myarray.set(2, 10);`
 - c) `myarray(2) = 10;`
 - d) `myarray[2] = 10;` ✓
 - ▶ Which of the following code fragments will NOT display all the elements in an array (names) ?
 - ▶ `for (String name: names)`
 - ▶ `System.out.println(name);`
 - ▶ `for (int index = 0; index < names.length; index++)`
 - ▶ `System.out.println(names[index]);`
 - ▶ `System.out.println(names);` ✓
-



Agenda & Reading

► Topics:

- Introduction
- Built-In Classes in Java
- Creating Classes
- Class Variables & Methods

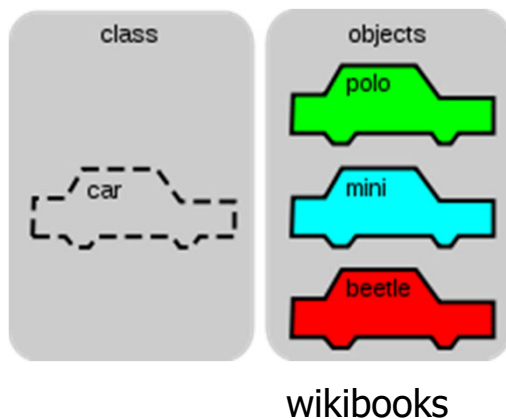
► Reading

- Java how to program Late objects version (D & D)
 - Chapter 7
- The Java Tutorial
 - Classes:
 - <http://docs.oracle.com/javase/tutorial/java/javaOO/classes.html>
 - Objects
 - <http://docs.oracle.com/javase/tutorial/java/javaOO/objects.html>

1. Introduction



- ▶ In object-oriented programming, a class is a **blueprint** for creating objects (a particular data structure), providing *initial values* for state (member variables or attributes), and implementations of *behavior* (member functions or methods).
- ▶ The class is a blueprint that defines a nature of a future object.
- ▶ An instance is a specific object created from a particular class.
- ▶ There may be zero, one, or many objects (instances) of a class.



Each instance, or object, has its own property values that may differ from those of other objects of the same kind



Nissan
Leaf
2010
35 km/h
4-door
Closed



Bentley
Hunaudières
1999
0 km/h
2-door
Closed



Ford
Mustang
2008
0 km/h
2-door
Open



2. Built-In Classes in Java

- ▶ Java provide some useful classes in the java.awt package.
 - ▶ These classes provide us with some methods and fields that help us save time and effort in writing code and repeating it more and more...
 - ▶ Example:
 - ▶ Point class :A point representing a location in (x,y) coordinate space, specified in integer precision.
 - Fields: x-coordinate, y-coordinate
 - ▶ Rectangle class: A rectangle specifies an area in a coordinate space that is defined by the rectangle's top-left point (x, y) in the coordinate space, its width, and its height.
 - Fields: top-left point (x,y), width and height

- ▶ Creating Objects from Java built-in classes

- ▶ Declaration
- ▶ Instantiation
- ▶ Initialization

<u>Point</u>	<u>Rectangle</u>
x =23	x =0
y = 94	y = 0
	width=50
	height=100



2. Built-In Classes in Java

Creating Objects

► Declaration:

```
Point p;
```

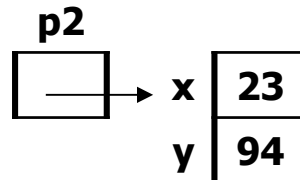
No object has been created with this declaration.

► Declaring a Variable to Refer to an Object

► Note: a variable either holds a primitive type, or it holds a reference to an object.

► Instantiation & Initialization:

```
Point p2 = new Point(23, 94);
```



- The new keyword is a Java operator that creates the object. This is known as instantiating a class.
- The **new** operator allocates memory for the new object
- The **new** operator returns a reference to the object it created
- The new operator is followed by a call to a constructor.
- For example, Point(23, 94) is a call to Point's only constructor. The constructor initializes the new object with x is 23 and y is 94



2. Built-In Classes in Java

Examples:

L08Code.java

► Objects: originOne, rectOne, rectTwo

```
import java.awt.Point;  
import java.awt.Rectangle;
```

Remember to add the
import statements

```
public class L08Code {  
    public static void main(String[] args){  
        Point originOne = new Point(23, 94);  
        Rectangle rectOne = new Rectangle(10, 20, 100, 200);  
        Rectangle rectTwo = new Rectangle(50, 100);  
        System.out.println(rectOne.getWidth());  
        System.out.println(rectTwo.getLocation());  
        System.out.println(originOne.toString());  
    }  
}
```

```
100.0  
java.awt.Point[x=0,y=0]  
java.awt.Point[x=23,y=94]
```

originOne:
Point

x =23

y = 94

rectOne:
Rectangle

x =10

y = 20

width=100

height=200

rectTwo:
Rectangle

x =0

y = 0

width=50

height=100



3. Creating Classes

- ▶ We can write our own classes that define specific objects that we need.
 - ▶ Each class you create becomes a new type that can be used to declare variables and create objects.
- ▶ A class definition has two parts: a class declaration and a class body.
 - ▶ Class declaration
 - ▶ contains keyword “class” followed immediately by the class’s name.
 - ▶ Class body
 - ▶ contains
 - constructors for initializing new objects,
 - declarations for the variables that provide the state, and
 - methods to implement the behavior of the class and its objects.
- ▶ Note: Each class declaration that begins with the access modifier **public** must be stored in a file that has the same name and ends with the .java extension.
- ▶ Class, method and variable names are identifiers. (i.e. all use camel case names.)

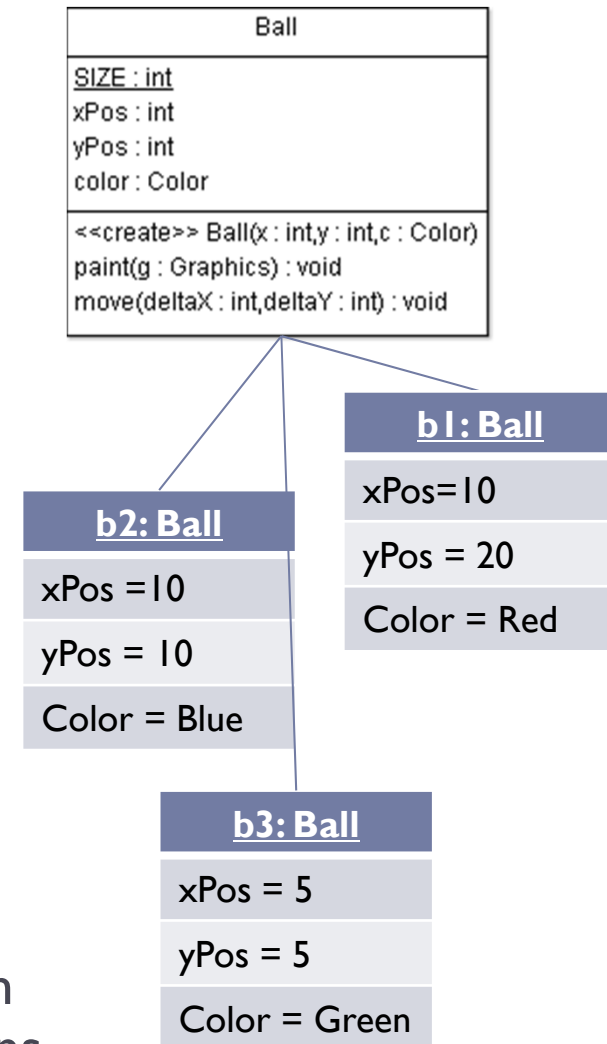
```
class MyClass {  
    //field and method declarations  
}
```



3. Creating Classes

Instance Variables

- ▶ Instance variables are dynamically allocated,
 - ▶ So they may have **different values** in **each** instance of an object.
 - ▶ A class declares the type of the data, but it does not reserve any memory space for it
 - ▶ When an object is instantiated, the runtime system allocates some memory to this instance – so that it can “**remember**” the values it stores in instance variables carried with it throughout its lifetime.
 - ▶ Instance variables exist before methods are called on an object.
 - ▶ A class normally contains one or more methods that **manipulate** the instance variables that belong to particular objects of the class.
 - ▶ Instance variables are declared **inside** a class declaration but **outside** the bodies of the class’s method declarations.





3. Creating Classes Instances of Car

Car.java

```
public class Car {  
    private String make;  
    private String model;  
    private int year;  
    private int speed;  
    private int doors;  
    private boolean openBonnet;  
  
    ...  
}
```

```
Car c1 = new Car("Nissan", "Leaf", 2010, 4);  
Car c2 = new Car("Bentley", "Hunaudieres", 1999, 2);  
Car c3 = new Car("Ford", "Mustang", 2008, 2);
```

► Pictures:



Nissan
Leaf
2010
35
4
false

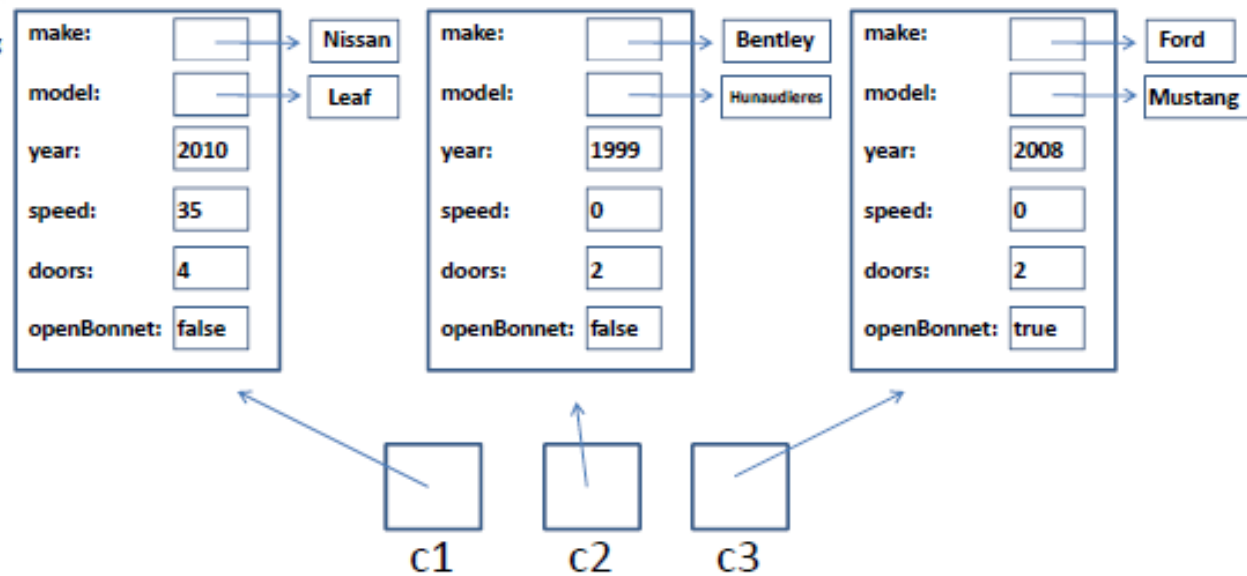
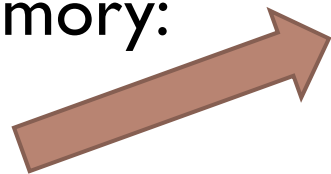


Bentley
Hunaudieres
1999
0
2
false



Ford
Mustang
2008
0
2
true

► Objects in memory:





3. Creating Classes

Visibility Modifiers

- ▶ Java provides a number of **access modifiers** to set access levels for classes, variables, methods and constructors. There are four levels but we look at two levels first.
 - ▶ Members of a class that are declared with **public** visibility can be accessed from anywhere
 - ▶ Members of a class that are declared with **private** visibility can only be access from inside the class
- ▶ As a general rule
 - ▶ No object's data should be declared as public.
 - ▶ Methods are usually declared as public so that they can be invoked by the clients
 - ▶ A method created simply to assist a service method is called a support method and should not be declared as public



3. Creating Classes

Constructors

- ▶ Constructors are invoked to **create** objects from the class blueprint and initialize instance variables.
- ▶ It looks like method declarations — except that they use the **name** of the class, can specify parameters but **no** return type.
- ▶ It automatically called when an object is created using the `new` operator
 - ▶ Note: Java requires a constructor call for every object that's created.
- ▶ It could have **more than one** constructors (i.e. overloading)
 - ▶ Each constructor has a different signature (a different number of sequence of argument types)

```
public Point() { ...  
public Point(Point p) { ...  
public Point(int x, int y) { ...
```

```
public Point() {  
    x=0;  
    y=0;  
}
```



3. Creating Classes Constructors

```
public Point() { }
```

default
constructor

► Default Constructor

- If a class definition doesn't include a constructor method, the Java compiler inserts a **default constructor** with no parameters, and the class's instance variables are initialized to their default values.
 - int:0, String: null, boolean: false
 - Recall that local variables are not initialized by default.
- Note: There's **No Default Constructor** in a Class That Declares a Constructor
 - If you have declared a constructor for a class, the compiler will **NOT** create a default constructor for that class.

```
public class MyMessage {  
    private int x;  
    private String y;  
}
```

m: MyMessage

x = 0

y = null

```
MyMessage m = new MyMessage();
```

```
public class Author {  
    private String name;  
    public Author(String name) {  
        this.name = name;  
    } ...  
}
```

```
Author a = new Author();
```





3. Creating Classes

Instance Methods

- ▶ Instance methods operate on this object's instance variables.
 - ▶ The operations that can be performed on each object may modify the **property values**.
 - ▶ We typically perform an operation on one object at a time.
- ▶ Similar to a class, a method definition has two parts: a method declaration and a method body.
 - ▶ A method declaration names the method's access level, return type, name, arguments.
 - ▶ The arguments specifies the type and name of each parameter
 - ▶ The return type indicates the type of value that the methods sends back to the calling location
 - ▶ A method that does not return a value has a void return type

**Access
Level**

Return Type

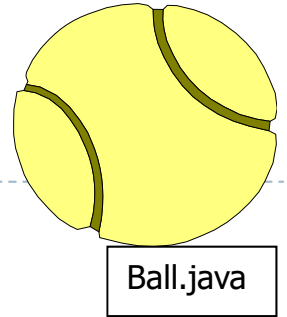
Arguments

```
public void move(int deltaX, int deltaY) {  
    ...  
}
```



3. Creating Classes

Example 1: Ball



```
public class Ball {  
    public final static int SIZE = 20;  
    private int xPos;  
    private int yPos;  
    private Color color;  
  
    public Ball(int x, int y, Color c) {  
        xPos = x;  
        yPos = y;  
        color = c;  
    }  
    public void move(int deltaX, int deltaY) {  
        xPos += deltaX;  
        yPos += deltaY;  
    }  
    ...  
}
```

Attributes/Fields

- ▶ Represent the internal state of an instance of this class.

Constructor

- ▶ Creates the object

Methods

- ▶ Implement the processing performed by or to an object, often updating its state.
- ▶ If there are read and write methods for an attribute **x**, these **should** be called **getX()** and **setX()**.
 - ▶ You should learn Java's conventions for capitalisation and naming.



3. Creating Classes

Object Instantiation

- ▶ When a constructor method is called, a new instance is created.

```
Ball b = new Ball( 10, 20, Color.Red );  
Ball c = new Ball( 0, 10, Color.Blue );
```

<u>c: Ball</u>
xPos = 0
yPos = 10
Color = Blue

<u>b: Ball</u>	
xPos = 10	→ 60
yPos = 20	→ 120
Color = Red	

- ▶ Message Passing

- ▶ In a method call, a message is passed to a **receiver** object.
- ▶ The receiver's response to the message is determined by its class.

```
b.move(50, 100);
```

receiver message arguments

```
public class Ball {  
    ...  
    public void move(int deltaX, int deltaY) {  
        xPos += deltaX;  
        yPos += deltaY;  
    }  
}
```

Exercise 1-2

- ▶ Define a class named **Person** which represents a person. It contains:

- ▶ private String name, private int age
- ▶ constructors x 2
- ▶ toString() method

```
Person p1 = new Person("Michael", 21);  
System.out.println(p1);  
Person p3 = new Person();  
System.out.println(p3);
```

```
Michael (21)  
(1)
```

- ▶ Define a class named **SimpleCircle** which represents a circle. It contains:

- ▶ private int radius
- ▶ constructors x 2
- ▶ toString() method

```
SimpleCircle p = new SimpleCircle();  
System.out.println(p);  
SimpleCircle p2 = new SimpleCircle(10);  
System.out.println(p2);
```

```
Circle, radius=1  
Circle, radius=10
```

Exercise 3-4: MyPoint class

- Define a class named MyPoint which represents a point object.

```
class MyPoint {  
    private int x;  
    private int y;  
    public MyPoint() {  
        //complete this  
    }  
    public MyPoint(int x, int y) {  
        //complete this  
    }  
}
```

```
MyPoint p = new MyPoint(10, 20);  
System.out.println(p);  
MyPoint p2 = new MyPoint();  
System.out.println(p2);
```

```
(10, 20)  
(0, 0)
```

- Consider the MyPoint class again, what will happen if the two constructors are declared as private?

```
class MyPoint {  
    private MyPoint() {  
        ...  
    }  
    private MyPoint(int x, int y) {  
        ...  
    }  
}
```



3. Creating Classes

```
System.out.println(b.color);
```



Instance Method: setXXX & getXXX

- ▶ Method `getX()` returns a particular `Ball` object's x-position to the caller.
 - ▶ This method has an empty parameter list, so it does not require additional information to perform its task and just return the value
 - ▶ Get methods are also commonly called accessor methods
- ▶ Method `setX()` assigns the value of the parameter (an `int`) to the class's `xPos` instance variable, thus storing the x-position in the object.
 - ▶ Note: If a method contains a **local variable** with the **same** name as an **instance variable**, that method's body will refer to the local variable rather than the instance variable.
 - ▶ The local variable is said to **shadow** the instance variable in the method's body.
 - ▶ You use the keyword **this** to refer to the shadowed instance variable explicitly
 - ▶ Set methods are also commonly called mutator methods

```
public int getX() {  
    return xPos;  
}
```

```
public void setX(int x) {  
    xPos = x;  
}
```

```
public void setX(int xPos) {  
    this.xPos = xPos;  
}
```



3. Creating Classes

Using the this Keyword

```
Point b = new Point( 10, 20);
```

b: Point
x =
y =

- ▶ Within an instance method or a constructor, “**this**” is a reference to the **current** object — the object whose method or constructor is being called.

- ▶ Using “this” with a Field

- ▶ When an instance variable and local variable share the same name, you must differentiate between the two by prefixing the instance variable access with the use of this.

Either `this.x = a;`
Or
`x = a;`

```
public Point(int a, int b) {  
    this.x = a;  
    y = b;  
}
```

```
public Point(int x, int y) {  
    this.x = x;  
    this.y = y;  
}
```

Must use **this.x** to refer to the instance field

- ▶ Using “this” with a Constructor

- ▶ One constructor can call another by using “this”

```
public Point() {  
    this(0,0);  
}
```



Case Study: Time

```
public class Time {  
    private int hour;    // 0 - 23  
    private int minute; // 0 - 59  
    private int second; // 0 - 59  
    ...  
}
```

► Case Study: Time class

► 5 overloaded constructors

- Overloaded constructors enable objects of a class to be initialized in different ways.
- To overload constructors, simply provide multiple constructor declarations with different signatures.

- The compiler **invokes** the **appropriate** constructor by matching the **number, types** and **order of the types** of the arguments specified in the constructor call with the number, types and order of the types of the parameters specified in each constructor declaration.

```
Time t1 = new Time();           // 00:00:00  
Time t2 = new Time( 2 );        // 02:00:00  
Time t3 = new Time( 21, 34 );   // 21:34:00  
Time t4 = new Time( 12, 25, 42 ); // 12:25:42  
Time t5 = new Time( 27, 74, 99 ); // 00:00:00  
Time t6 = new Time( t4 );        // 12:25:42
```



Case Study: Time

Overloaded Constructors

- ▶ Such a constructor simply initializes the object as specified in the constructor's body
- ▶ Using “this” in method-call syntax as the first statement in a constructor's body **invokes another constructor** of the same class.
- ▶ Popular way to reuse initialization code provided by another of the class's constructors rather than defining similar code in the no-argument constructor's body.
- ▶ Once you declare any constructors in a class, the compiler will not provide a default constructor.
- ▶ Standard constructor:

```
public Time() {  
    this( 0, 0, 0 );  
}
```

Time()

Time(int h)

Time(int h, int m)

Time(int h, int m, int s)

Time(Time t)

```
public Time(int h, int m, int s){  
    setTime( h, m, s );  
}
```

```
public Time( int h, int m ){  
    this( h, m, 0 );  
}
```

```
public Time(Time t){  
    this(t.hour, t.minute, t.second );  
}
```



Exercise 5

- What is the output?

```
public class Puppy{
    private int puppyAge=1;
    private String puppyName;
    public Puppy(String name){ puppyName=name; }
    public void setAge( int age ){
        puppyAge = age;
    }
    public int getAge( ){
        return puppyAge;
    }
    public static void main(String []args){
        Puppy myPuppy = new Puppy("tommy");
        myPuppy.setAge( 2 );
        int value = myPuppy.getAge( );
        System.out.println("Variable Value :" + myPuppy.puppyAge );
    }
}
```




Exercise 6-8



- ▶ Consider the Person class defined from above, add the following:
 - ▶ The accessor and mutator methods
 - ▶ getName()
 - ▶ getAge()
 - ▶ setName(String name)
 - ▶ setAge(int age) Note: only modify the age if the range is between 0 and 120
 - ▶ A method named growOder() which increases age by 1
 - ▶ A method named isOfLegalAge() which returns true if age ≥ 18
- ▶ Fields: weight and height
- ▶ A constructor
- ▶ The accessor and mutator methods
- ▶ A method named getBodyMassIndex() which returns the BMI of a person
- ▶ Modify the toString() method

Exercise 9

- ▶ Consider the `MyPoint` class defined from above, add the following:
 - ▶ The accessor and mutator methods
 - ▶ `getX()`
 - ▶ `getY()`
 - ▶ A method named `add(MyPoint other)`
 - ▶ returns a new `MyPoint` object that contains the sum of the x coordinates and the sum of the y coordinates.
 - ▶ A method named `multiply(MyPoint other)`
 - ▶ returns the dot product of the two points, defined according to the rules of linear algebra. The formula is $x * other.x + y * other.y$
 - ▶ A method named `scalar_multiply(int value)`
 - ▶ returns a new `MyPoint` whose coordinates are a multiple of the original coordinates.



4. Class Variables & Class Methods

Class Variables

- ▶ You specify a class member variable or a class method by using the **static** keyword in the member's declaration.
- ▶ A static variable represents classwide information—all objects of the class share the same piece of data.
- ▶ Class variables are statically allocated, so they
 - ▶ are **shared** by an **entire Class of objects**
 - ▶ The runtime system allocates class variables once per class, regardless of the number of instances created of that class. (i.e. only one copy)
 - ▶ Static storage is allocated when the class is loaded.
 - ▶ All instances **share** the same copy of the class variables

```
public class Stuff {  
    public static int x = 10;  
}
```

```
Stuff a1 = new Stuff();  
Stuff a2 = new Stuff();  
System.out.println(Stuff.x);
```

Stuff

X: 10

a1: Stuff

a2: Stuff



4. Class Variables & Class Methods

Class Methods

- ▶ Some methods do work but don't need an object. Use the class name instead of an object name to invoke

```
Math.round(2.3) ;  
Integer.parseInt("123") ;
```

- ▶ They are called class methods.
- ▶ Declare class methods with the **static** modifier
- ▶ Class methods cannot access instance variables.
- ▶ Class methods are handled by the “class object” – they can be called even if there are no instances of this class.

```
public class Stuff {  
    private static int x = 10;  
    public static int getX() { ...  
    }  
}
```

```
System.out.println(Stuff.getX());
```



4. Class Variables & Class Methods

Additional Notes

- ▶ A static method can call other static methods of the same class directly (i.e., using the method name by itself) and can manipulate static variables in the same class directly.
- ▶ Java does not allow a static method to directly access instance variables and instance methods of the same class
 - ▶ It is because a static method can be called even when no objects of the class have been instantiated. (i.e. *this* reference cannot be used in a *static* method)
- ▶ Instance methods can access all fields (static variables and instance variables) and methods of the class.
 - ▶ Many objects of a class, each with its own copies of the instance variables, may exist at the same time.
- ▶ Declaring instance variables private is known as data hiding or information hiding.

Examples: Class & Instance variables

```
public class SharedCounter {  
    private static int count;  
    private int value;  
    public SharedCounter(int value) {  
        this.value = value;  
        count++;  
    }  
    public String toString() {  
        return "value=" + value + " count=" + count;  
    }  
}
```

Class
variable

Instance
variable

```
SharedCounter sc1 = new SharedCounter(10);  
SharedCounter sc2 = new SharedCounter(100);  
SharedCounter sc3 = new SharedCounter(200);  
System.out.println(sc1 + " " + sc2 + " " + sc3);
```

: Class

name = "SharedCounter"
count = 0>1>2>3



c1: SharedCounter

value =10

c2: SharedCounter

value =100

c3: SharedCounter

value =200



Exercise 10

```
Employee bob = new Employee("Bob", "Blue");  
Employee susan = new Employee("Susan", "Baker");  
System.out.println(Employee.getCount());
```

► Complete the following Employee class:

2

- Add a static variable to maintain a count of the number of Employee objects in memory.
- Complete the constructor
- Add a static getCount() method



```
class Employee {  
    private String firstName;  
    private String lastName;  
    // complete this  
  
    public Employee( ...  
  
    }  
  
    public String toString() {  
        return String.format("%s, %s\n", lastName, firstName);  
    }  
}
```



enum - methods

```
enum Size {  
    SMALL, MEDIUM, LARGE, EXTRALARGE;  
    public int getSize() { ...return ...}  
}
```

- ▶ an enum is a type that has a fixed set of constant values.
- ▶ enum types are considered to be a special type of class
 - ▶ An enum class can include methods and fields.
- ▶ Some predefined methods in enum classes that are re `public int static final SIZE`
 - ▶ `ordinal()`: returns the position of an enum constant.
 - ▶ `toString()`: returns the string representation of the enum constants
 - ▶ `valueOf(String s)`: takes a string and returns an enum constant having the same string name
 - ▶ `values()`: method returns an array of enum type containing all the enum constants

```
Size s1 = Size.SMALL;  
System.out.println(s1.ordinal());  
System.out.println(s1.toString());  
System.out.println(Size.values()[2]);  
Size s = Size.valueOf("MEDIUM");  
System.out.println(s);  
System.out.println(Arrays.toString(Size.values()));
```

```
0  
SMALL  
LARGE  
MEDIUM  
[SMALL, MEDIUM, LARGE, EXTRALARGE]
```




enum Constructors

- ▶ an enum class may include a constructor like a regular class
- ▶ but define as private (i.e. accessible within the class)

```
enum Size {  
    SMALL("The size is small."),  
    MEDIUM("The size is medium."),  
    LARGE("The size is large."),  
    EXTRALARGE("The size is extra large.");  
    private final String pizzaSize;  
  
    private Size(String pizzaSize) { this.pizzaSize = pizzaSize;}  
  
    public String getSize() {return pizzaSize;}  
}
```

```
Size size = Size.SMALL;  
System.out.println(size.getSize());  
  
Size s2 = Size.LARGE;  
System.out.println(s2.getSize());
```

```
The size is small.  
The size is large.
```



Exercise 11

- ▶ Consider an assignment, the full mark is 10, the passing mark is 5.
 - ▶ If a student obtains
 - ▶ 9 or 10 scores, the grade is A; 7 or 8, the grade is B; 5 or 6, the grade is C
 - ▶ 3 or 4, the grade is D; 0 or 1 or 2 the grade is E.
 - ▶ Define an enum named Grade, the constants are: E, D, C, B, A
 - ▶ Add a static method named `convertToGrade(int score)` which takes an integer score as a parameter and converts to a grade based on the above scheme.
 - ▶ Add a method which returns a boolean. The method returns true if the grade is A or B or C, and returns false otherwise

```
enum Grade { E, D, C, B, A;  
    public static Grade convertToGrade(int score) {  
        if (score > 8) return Grade.A;  
        else if (score > 6) ...  
    }  
    public boolean isPass() { return ordinal() >= Grade.C.ordinal(); }  
}
```

```
Grade myGrade = Grade.convertToGrade(7);  
System.out.println(myGrade.isPass());  
Grade yourGrade = Grade.convertToGrade(4);  
System.out.println(yourGrade.isPass());
```

true
false